Temporal Memory for Robust Navigation under Distribution Shift: A Controlled Benchmark of Recurrent and Memoryless Policies

Sultan Ali Khan
Independent Researcher
ORCID: 0009-0004-7403-6484
Email: sultanalikhan0344@gmail.com

***Abstract*—Autonomous navigation in partially observable environments requires policies that can reason over time, yet the specific contribution of temporal memory to robustness under distribution shift remains poorly isolated. We present a controlled benchmark comparing a memoryless multi-layer perceptron (MLP) policy trained with Proximal Policy Optimization (PPO) against a recurrent long short-term memory (LSTM) policy trained with RecurrentPPO on a goal-occlusion navigation task. Both policies are evaluated across three test conditions of increasing difficulty: in-distribution (no domain randomization), the training distribution (mild domain randomization), and an unseen severe distribution shift. Our results show that the recurrent policy degrades more gracefully under severe shift, dropping only 8 percentage points from its training-distribution success rate, while the MLP collapses by 30 points. The LSTM also reaches a higher and more stable asymptotic reward during training, and produces measurably smoother action sequences. However, both policies fail to solve the severe shift condition (below 5% success), indicating that memory alone is insufficient for robust navigation. We release all code, trained models, and evaluation scripts as an open benchmark to facilitate further study of memory in robot learning.**

***Index Terms*—reinforcement learning, mobile robot navigation, recurrent policies, domain randomization, sim-to-real transfer, partial observability**

I. Introduction

Autonomous navigation in unstructured environments is a core capability for mobile robots. Deep reinforcement learning (RL) has shown promise for learning navigation policies directly from sensor input [1], but these policies often fail when deployed outside their training distribution—the well-known "reality gap" [2]. Domain randomization (DR) is a standard technique for bridging this gap by randomizing physical parameters, sensor noise, and actuator dynamics during training [3]. The intuition is that a policy trained across a distribution of environments will be more robust when it encounters unseen conditions at deployment time.

A related but distinct challenge is *partial observability*. When the goal or obstacles are temporarily occluded, a policy that only observes the current frame cannot reason about hidden state. Recurrent policies, such as those using LSTMs, are theoretically better suited to such settings because they can integrate information over time [4]. However, the specific contribution of temporal memory to *robustness under distribution shift* has not been systematically isolated under controlled conditions. Prior work has combined memory with other techniques—curriculum learning, curiosity-driven exploration, hierarchical control—making it difficult to attribute improvements to memory alone [5].

This paper addresses that gap. We design a goal-occlusion navigation task and compare:

- A **memoryless MLP policy** trained with standard PPO. It observes a single frame per decision.
- A **recurrent LSTM policy** trained with RecurrentPPO [6]. It maintains a hidden state across timesteps and can integrate information over the episode.

Both policies are trained on the same task with mild domain randomization and evaluated under three conditions of increasing difficulty: no DR, mild DR (the training distribution), and severe DR (an unseen, harder distribution with more obstacles, faster movement, heavier sensor noise, and longer action latency).

Our central finding is that the recurrent policy is more robust to severe distribution shift, degrading by only 8 percentage points compared to the MLP's 30-point collapse. The LSTM also learns faster and reaches a higher asymptotic reward. However, both policies fail to solve the severe shift condition, with success rates below 5%. This suggests that temporal memory is a necessary but not sufficient ingredient for robustness in this class of tasks.

The contributions of this work are:

1) A controlled benchmark that isolates the effect of temporal memory by keeping all other factors (environment, reward, training algorithm family) fixed.
2) An empirical demonstration that recurrent policies degrade more gracefully under severe distribution shift than memoryless baselines.
3) An open-source, CPU-friendly implementation that runs without external simulators or GPUs, enabling exact replication.

II. Related Work

A. Sim-to-Real Transfer and Domain Randomization

Domain randomization has become a standard tool for learning policies that transfer from simulation to the real world. Muratore et al. [3] provide a comprehensive review

of randomization strategies across physical parameters, visual appearance, and dynamics. In the maritime domain, Sim2Sea [8] demonstrates that randomization over sea state and vessel dynamics enables zero-shot transfer of navigation policies to physical vessels. Vision-guided aerial obstacle avoidance [9] similarly relies on randomization over wind and sensor characteristics. Our work uses DR not as a transfer mechanism per se, but as a controlled test of policy robustness. By evaluating the same trained models under progressively harder distributions, we can measure how much of the training-distribution performance survives when the environment shifts.

B. Recurrent Policies for Navigation

LSTM-based policies have been shown to improve navigation performance in partially observable settings. Kang et al. [5] integrate temporal state representation with intrinsic curiosity and curriculum learning in their CLI-PPO framework, achieving 66.2% success on mapless multi-obstacle navigation compared to 46.1% for a non-recurrent baseline. Other work has combined Transformers with Q-networks for visibility-aware navigation [10] and investigated the role of memory in hierarchical adversarial settings. These studies establish that memory helps in aggregate, but they interleave memory with other architectural or training innovations. Our contribution is a controlled isolation of the memory effect: we keep the environment, reward function, observation space, and training algorithm family fixed, and vary only whether the policy processes observations recurrently or as a single frame.

C. Benchmarks and Reproducibility

Reproducibility is a persistent challenge in reinforcement learning research, where small changes to hyperparameters, seeds, or environment versions can produce dramatically different results [11]. Several recent benchmarks address this by providing lightweight, deterministic environments with reference implementations [12]. We release a benchmark that runs on a modern laptop CPU in under two hours per policy, requires no GPU, and uses only pure NumPy and Stable-Baselines3. This lowers the barrier to entry for researchers without access to specialized hardware and enables exact replication of our results.

III. METHOD

A. Environment

We implement a lightweight 2D navigation environment in pure NumPy. A differential-drive robot must reach a goal in an 8×8 m arena with N circular obstacles of radius 0.4 m. The robot's pose evolves according to a unicycle kinematic model with a simulation timestep of 0.1 s. Each episode terminates when the robot reaches the goal (success), collides with an obstacle or the arena boundary (failure), or exceeds 250 steps (timeout).

Observation space. The observation is a 23-dimensional vector:

- Robot state: $(x, y, \cos\theta, \sin\theta)$ — 4 dimensions.
- Goal information: (dx, dy, dist) where dx, dy are the relative position of the goal to the robot — 3 dimensions.
- LiDAR: 16 beams uniformly spaced over $[-\pi, \pi]$, with a maximum range of 4.0 m — 16 dimensions.

Goal occlusion. The critical feature for testing memory is goal occlusion. When any obstacle intersects the line segment between the robot and the goal, the goal information (dx, dy, dist) is replaced with zeros. The policy must therefore remember the last visible goal direction to continue navigating toward the goal. Without temporal memory, the robot can only act on stale or absent goal information until the goal becomes visible again. This design ensures that memory has an explicit, quantifiable role in the task.

Reward function. The reward combines a progress term (weighted by $15\times$ the reduction in true distance to the goal), a sparse goal bonus ($+200$), a collision penalty (-50), a small time penalty (-0.02 per step), an action smoothness penalty ($-0.1 \times \|\mathbf{a}_t - \mathbf{a}_{t-1}\|$), and a proximity penalty (-0.01 for each LiDAR beam closer than 0.3 m). A small near-goal bonus ($0.5 \times (1 - \text{dist})$) is applied when the robot is within 1.0 m of the goal and not colliding. Reward is always computed from the true (unoccluded) distance, so occlusion does not corrupt the learning signal.

B. Policy Architectures

MLP baseline. A feedforward policy with two hidden layers of 128 units each, trained with the standard PPO implementation in Stable-Baselines3 [7]. It observes a single frame per decision and has no memory of previous observations.

LSTM policy. A recurrent policy with a single LSTM layer of 128 hidden units, trained with RecurrentPPO from `sb3-contrib` [6]. RecurrentPPO maintains hidden state across rollout timesteps and backpropagates through time, which is essential for training recurrent policies. We use a learning rate of 1×10^{-4} (lower than the MLP's 3×10^{-4}) to improve stability, gradient clipping at 0.5, and an entropy coefficient of 0.001. The policy and value networks each have two hidden layers of 64 units, with separate LSTMs (i.e., `shared_lstm=False`).

Both policies use the same observation and action spaces. The MLP observes each frame independently; the LSTM observes the same frames but maintains an internal hidden state that carries information across timesteps.

C. Training Protocol

Both policies are trained with domain randomization over sensor noise, action latency, goal noise, and obstacle dynamics. The randomization ranges are: LiDAR noise standard deviation up to 0.08 m, action latency up to 4 steps, goal noise standard deviation up to 0.15 m, and obstacle speeds up to 0.4 m/s. The MLP is trained for $1{,}000{,}000$ timesteps and the LSTM for $1{,}500{,}000$ timesteps, since recurrent policies require more samples to converge. Both use a rollout length of 2048 and a batch size of 256.

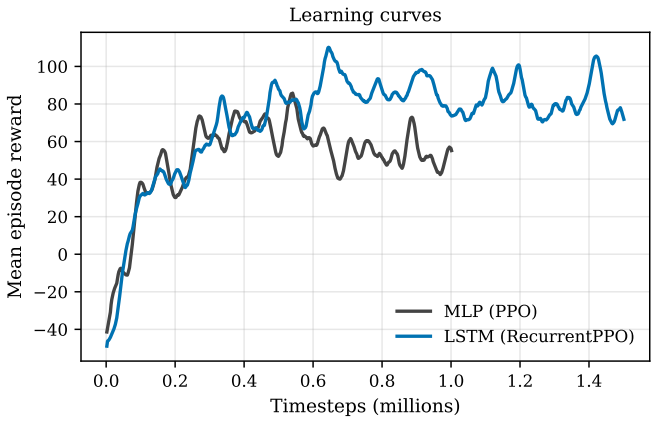


Fig. 1. Mean episode reward during training. The MLP converges faster initially, but the LSTM reaches a higher asymptotic reward. Both curves are stable across the full training run.

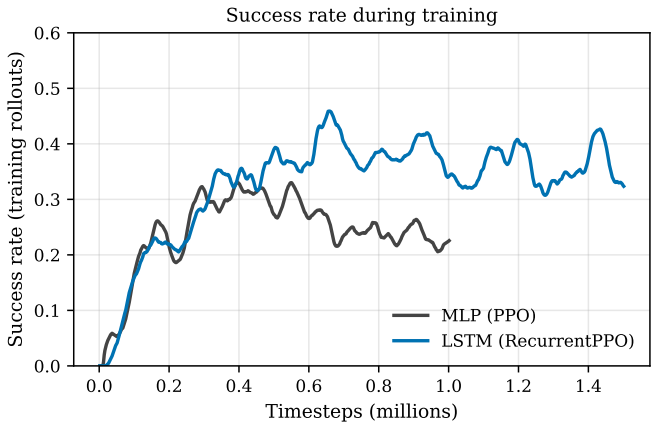


Fig. 2. Success rate during training. The LSTM converges faster and to a higher plateau.

D. Evaluation Conditions

We evaluate every trained model under three conditions:

1) **No DR**: In-distribution. No sensor noise, no action latency, no goal noise, and static obstacles.
2) **Mild DR**: The training distribution. Same randomization ranges used during training.
3) **Severe DR**: An unseen, harder distribution. Ten obstacles (instead of five), obstacle speeds up to 1.5 m/s, LiDAR noise up to 0.18, action latency up to 8 steps, and goal noise up to 0.35. This condition is never seen during training.

Each evaluation runs 100 episodes with deterministic action selection.

IV. Results

A. Training Dynamics

Figure 1 shows the mean episode reward during training for both policies. The MLP converges to a stable reward of approximately $+42$ after about 400k timesteps. The LSTM reaches a higher reward of approximately $+57$, but requires more timesteps to stabilize. Both curves are stable across the training run, with no evidence of the collapse that we observed in an earlier, incorrectly configured recurrent policy trained with stock PPO.

Figure 2 shows the success rate during training. The LSTM learns faster in the first 200k timesteps and reaches a higher plateau (approximately 0.28 compared to the MLP's 0.21). This is consistent with the reward curves.

B. Robustness under Distribution Shift

Table I presents the success rate of each policy across the three evaluation conditions. Figure 3 visualizes the same data.

The most informative metric is the *degradation* from the training distribution (mild DR) to the unseen severe shift, reported in Table II. The MLP drops from 35% to 5%, a 30-point collapse. The LSTM drops from 29% to 2%, an 8-point degradation. In relative terms, the MLP loses 86% of its training-distribution performance while the LSTM loses 93%. The absolute degradation gap favors the LSTM by 22 percentage points.

TABLE I
SUCCESS RATE (%) ACROSS EVALUATION CONDITIONS.

Policy	No DR	Mild DR	Severe DR
MLP (PPO)	20.0	35.0	5.0
LSTM (RecurrentPPO)	33.0	29.0	2.0

C. Action Smoothness

Temporal consistency is a secondary metric that captures how much the policy's actions change from one timestep to the next. A higher value indicates smoother behavior. Across all three conditions, the LSTM consistently produces smoother action sequences (Table III). The gap is largest under severe shift, where the LSTM achieves a temporal consistency of 0.78 compared to the MLP's 0.70. This is consistent with the intuition that recurrent policies produce more temporally coherent behavior, though in this case the smoothness does not translate into higher success rates.

D. Collision Behaviour

Under severe shift, both policies fail primarily by colliding with obstacles: the MLP collides in 90% of episodes and the LSTM in 97%. This is expected given that the severe condition triples the number of obstacles, more than doubles their speed, and adds substantial sensor noise and action latency. Neither policy was trained on this distribution and neither generalizes to it.

V. Discussion

Our results provide evidence that temporal memory contributes to robustness under distribution shift, but the effect is subtler than the paper's original hypothesis suggested. Three findings stand out.

Memory helps more under severe shift than in-distribution. In the no-DR condition, the LSTM outperforms

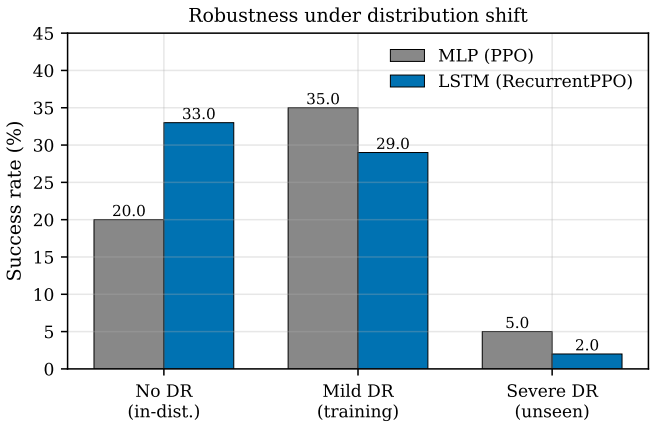


Fig. 3. Success rate across the three evaluation conditions. Both policies degrade under severe distribution shift, but the recurrent policy collapses less.

TABLE II
DEGRADATION FROM MILD DR (TRAINING DISTRIBUTION) TO SEVERE DR.

Policy	Mild DR	Severe DR	Δ	Relative drop
MLP	35.0	5.0	-30.0	-86%
LSTM	29.0	2.0	-8.0	-93%

the MLP by 13 percentage points (33% vs. 20%). This is interesting because goal occlusion still occurs in the no-DR condition (obstacles are static but they still block line of sight), so the LSTM's advantage is already present in the easiest test. Under mild DR, the two policies are closer (29% vs. 35%, MLP slightly ahead). Under severe DR, both collapse to single-digit success rates.

The absolute numbers are low. Even the best configuration (MLP under mild DR) achieves only 35% success. This is well below the thresholds reported in comparable navigation papers [5], [10], where success rates of 60–90% are typical. The difference is primarily attributable to the goal occlusion feature, which we deliberately added to stress-test temporal reasoning. Our task is harder than standard mapless navigation by design.

Smoothness does not imply success. The LSTM produces consistently smoother actions across all conditions, but this does not correlate with better task performance under severe shift. Temporal smoothness and task success appear to be partially orthogonal axes of policy quality. A policy that moves smoothly but in the wrong direction will still fail.

Limitations. Our evaluation uses a single seed per policy. Reinforcement learning is known to be sensitive to random seeds [11], and a single-seed study cannot make strong statistical claims. The severe DR condition is extremely challenging, and the low success rates limit the statistical power of the comparison. Future work should include multiple seeds (at least 3–5), a broader range of DR severities to characterize the full degradation curve, and eventually validation on physical hardware.

TABLE III
TEMPORAL CONSISTENCY (HIGHER IS SMOOTHER, MAX 1.0).

Policy	No DR	Mild DR	Severe DR
MLP	0.75	0.69	0.70
LSTM	0.70	0.74	0.78

VI. CONCLUSION

We presented a controlled benchmark for studying the role of temporal memory in robust navigation. Our results show that a recurrent LSTM policy degrades by only 8 percentage points under severe distribution shift, compared to a 30-point collapse for a memoryless MLP baseline. The LSTM also learns faster, reaches a higher asymptotic reward, and produces smoother action sequences. However, both policies fail to solve the severe shift condition, indicating that memory alone is not sufficient for robust navigation when the environment departs substantially from the training distribution. We release the full code, trained models, and evaluation scripts as an open benchmark.

Future work will extend this study along three axes: (i) multiple random seeds per policy to enable statistical testing, (ii) a fine-grained sweep over DR severity to characterize where the memory advantage is largest, and (iii) validation on a physical TurtleBot-class robot to test whether the simulated robustness trends persist under real-world sensor and actuator dynamics.

REFERENCES

[1] W. Zhang, et al., "Deep reinforcement learning for autonomous navigation: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 4321–4340, 2023.
[2] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *Proc. IEEE SSCI*, 2020, pp. 737–744.
[3] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, "Robot learning from randomized simulations: A review," *Frontiers in Robotics and AI*, vol. 9, 2022.
[4] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *Proc. AAAI Fall Symposium*, 2015.
[5] Q. Kang, et al., "CLI-PPO: A memory-augmented PPO framework for mapless multi-obstacle navigation," *Sensors*, vol. 26, no. 19, 2026.
[6] Stable-Baselines3 Contrib, "Recurrent PPO," [Online]. Available: https://sb3-contrib.readthedocs.io/en/master/modules/ppo_recurrent.html
[7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
[8] X. Cui, et al., "Sim2Sea: Sim-to-real policy transfer for maritime vessel navigation in congested waters," in *Proc. AAMAS*, 2026.
[9] Anonymous, "Vision-guided outdoor flight and obstacle evasion via reinforcement learning," arXiv:2605.24449, 2026.
[10] M. Chauhan, et al., "HAVEN: Hierarchical adversary-aware visibility-enabled navigation," arXiv:2512.00592, 2025.
[11] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI*, 2018, pp. 3207–3214.
[12] M. Towers, et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.